EIE2903/IC2141 - Module Code: TM1118

Module Title: Project with Internet and Multimedia

Project Report

Team B08

- Group Members -

Student Name	Student ID
Kit	[Redacted]
[Group Member 2]	[Redacted]
[Group Member 3]	[Redacted]

Contents

Group Section

1) Feature description

1.1 Introduction

The main objective of this project is developing a Smart Campus network at W311 and IoT web applications. We designed a home page for the web application, with a Smart Campus navigation menu directing users to other subpages. It contains hyperlinks of admin login page, log page, dashboard, Time-Event-Data page and Data Analysis page. Additionally, we created top and bottom navbars using headers and footers to enhance the accessibility and navigation of websites.

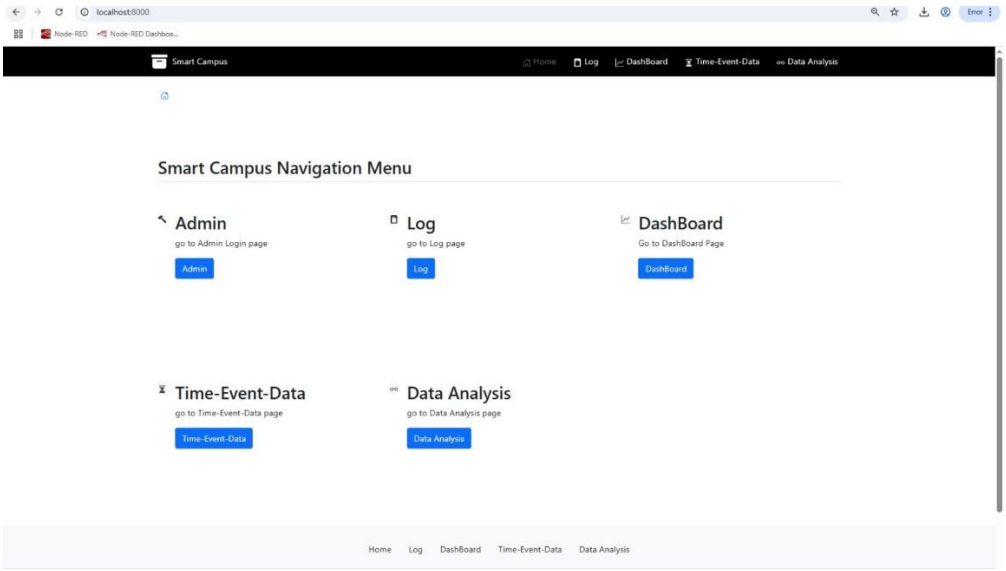

Figure 1: Home Page

1.2 IOT sensor nodes

First, we designed an IoT sensor node using a Raspberry Pi to detect and collect environmental data. It was installed in W311-Z2 to measure the environmental data in that venue. The sensor application will execute automatically upon start-up. It can also be controlled remotely by an ESP8266 IoT-status tag to switch between different modes for different function.

The default mode is the IoT sensor. When it is active, the LCD display shows a two-line message: “TEAM B08 Node” and “Now Operating!”, with a LED flashing to indicate its status. It publishes the MQTT sensor data in JSON format every five minutes, including the sensor node ID, location of the venue, temperature, relative humidity, light intensity (from 0-100%) and ambient sound level in decibel.

The second mode is a Run-Jump-Fall minigame, which was developed in the previous training module. When the game starts, the character starts running and the player needs to press the button to jump over the hole. The goal is to survive as long as possible to obtain higher scores.

The third mode is a Standby mode. The LCD display will show a message “Standby mode” to indicate the state of sensor. This allows the sensor to conserve energy when it is not actively being used and allows the user to stop data publishing without the need to shut the sensor down or stop the running program.

Apart from Raspberry Pi, we configured an M5-StickCPlus2 as an additional IoT sensor. Therefore, users are not required to turn on the Raspberry Pi to collect the data. When the device is switched on, the measured value of temperature and relative humidity will be displayed on the LCD panel. The IoT data will be published in the same JSON format as the RPI sensor. However, as the M5-StickCPlus2 is not able to measure light intensity and sound level, their default value is set as zero.

1.3 Data Log Page

The data log page displays all the sensor data in the database with different filters. The filters include node id, location, time, ranges of temperature, humidity, light intensity and sound level. After selecting filters, user clicks the “Apply filter” button below the filters field to display the filtered data log.

At the example below, users want to filter data collected by the A01 sensor node between 6pm on 7 August 2025 and 6pm on 9 August 2025, with a minimum temperature of 20 °C and a minimum light intensity of 30%. After applying the filters, the filtered data log will be displayed in table format.

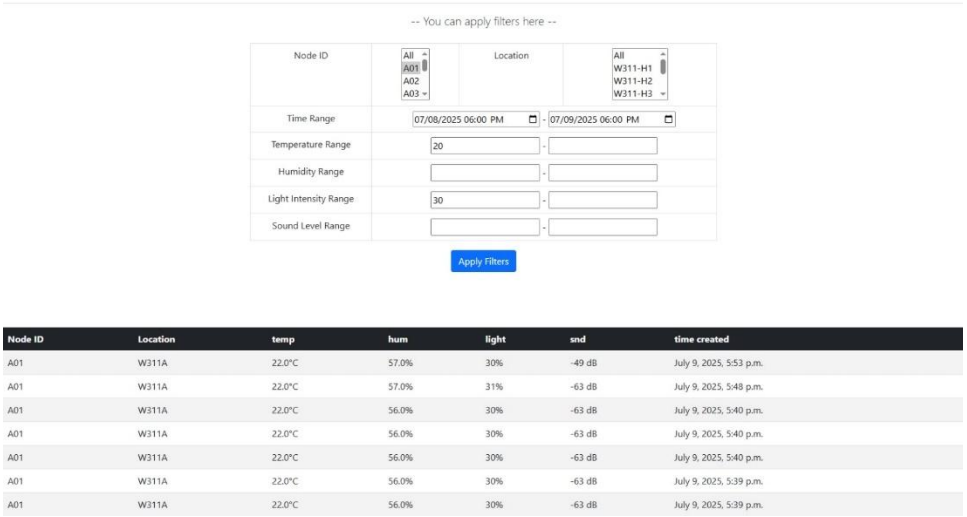

Figure 2: Data Log Page

1.4 Dashboard Page

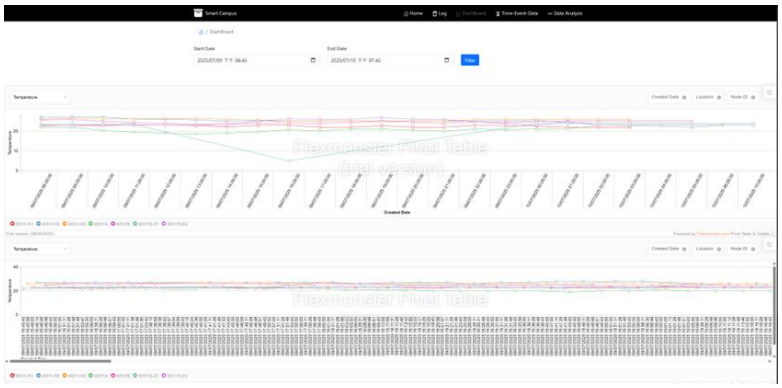

Figure 3: Dashboard Page

The IoT (Internet of Things) dashboard serves as a visual interface that aggregates, monitors, and manages data from connected IoT devices. It combines data streams from various devices into unified views. It features two charts: the upper chart displays the average of a specific environmental parameter for each hour, while the lower chart presents all recorded environmental parameters.

Users have the option to apply filters to the data by selecting the time range, location, and node ID. This allows users to focus on the information that is most relevant to their data analysis needs. When displaying data/records from two or more locations, users can see lines in different colors, enabling them to compare the environmental data of the locations.

Additionally, users can select environmental parameters, including temperature, humidity, light intensity, and sound level. For the upper chart, users can also select the event count, which shows the number of records within each hour.

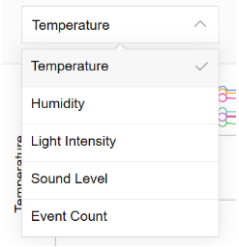

Figure 4: Choice for Environmental parameters (plus record count per hour)

To sum up, those functions serve as a starting point for future performance analytics, with a dashboard that visualizes trends. Users are provided with a brief description of the environmental data during different periods of time. With dynamic filtering capabilities to filter data by time range, location, and environmental parameters, enabling user for focused analysis. This facilitates efficient trend recognition, comparative assessment across devices or time periods, which provide vital information for security alerting and operational efficiency gains.

1.5 Time-Event-Data Page

For the Time-Event-Data page, the user will be faced with an option/filter selector. The user can select a venue, then select a time range, as well as apply filters regarding the range of temperature, humidity, light intensity, and sound level as shown in. After selecting the desired options, the user can click the “Apply Filters” button.

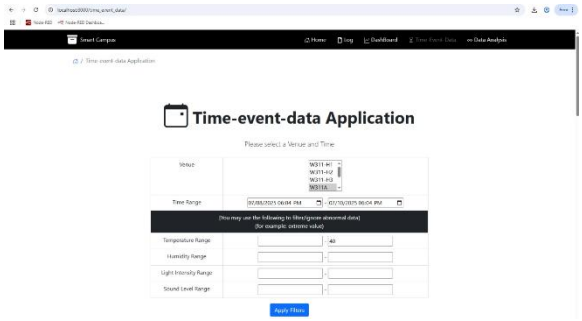

Figure 5: Time-Event-Data Page

After applying the filter, a Time Event List will be displayed in a table format, showcasing events that match the selected venue and overlap with the chosen time range. Each entry will provide details about the event, including the venue, time range, event code, and instructor. Below the Time Event List, there will be a summary table of environmental data for each event's time range, displaying the average, maximum, and minimum values for temperature, humidity, light intensity, and sound levels.

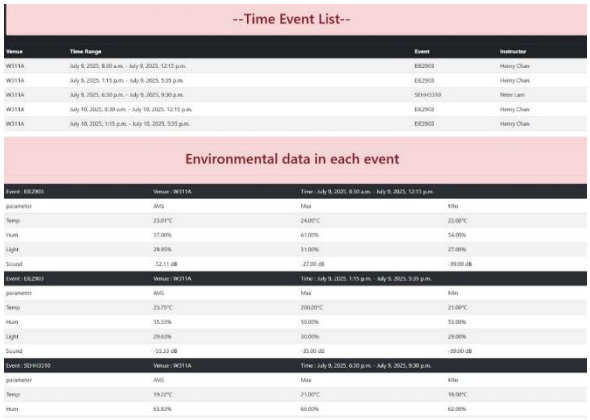

Figure 6: Time Event List & Environmental data table

Figure 7: Environmental data – Overall (without filter)

Figure 8: Environmental data – Overall (with filter)

At the bottom of the page, there is a table that presents the overall environmental data, summarizing the information for all events as shown in Figure 7 and Figure 8.

As shown in Figure 7, the maximum temperature is 200 degrees. This occurs because abnormal data records may arise from sensor errors or other reasons, such as a temperature reading of 200 degrees, which negatively impacts the summary of environmental data. This is why a filter, as shown in Figure 5, is designed. It allows users to select a valid range of data. After applying the filter, abnormal records will be ignored, enabling users to obtain accurate information, as shown in Figure 8, with a reasonable value of 24 degrees.

1.6 Data analysis and Alert Generations

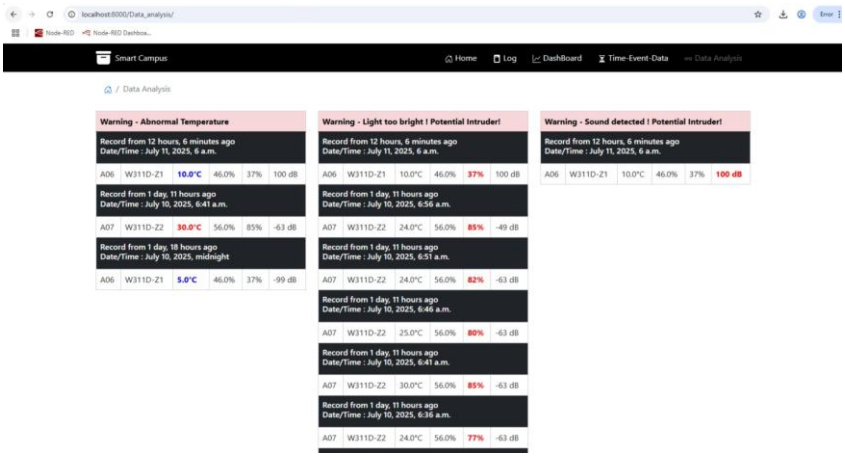

Figure 9: Data Analysis Page

For the data analysis page, it will analyze environmental data during nighttime (11:00 PM – 6:00 AM) and display warning alerts regarding abnormal temperature, light intensity, and sound levels. For each warning type, the corresponding data will be highlighted with color. Specifically, for temperature, if the temperature is abnormally cold (≤ 10 degrees), it will be displayed in blue, and if it is abnormally high (≥ 30 degrees), it will be displayed in red. The warning will be displayed in chronological order, and users will also be able to see when the record was made and how much time has passed.

Also, when an abnormal reading is detected, the information will be sent to the IoT sensor node. The node's buzzer will emit an alert sound for a few seconds and display a warning based on the abnormality (e.g., too hot, too cold, too bright, or too loud). This allows users to be informed and alerted of any abnormal records so they can address potential problems or intruders.

Figure 10: M5-StickCPlus2 sensor

Moreover, the M5-StickCPlus2 sensor supports abnormal reading detection and automatic alert generation. Users can turn the detection and alert system on or off by pressing either Button A or Button B on the M5Stick. A message will then appear to indicate the alert status. When the sensor obtains an abnormal reading, it will display a warning message based on the abnormality (e.g., too hot, too cold or too wet) and the buzzer will output an alert. The alert system only detects environmental data from local sensor nodes. This results in less delay between detection and alerting.

2) Design methodology

2.1 Smart Campus application

A “Smart_Campus” application is created in Django for developing the homepage, data log page and dashboard.

2.2 Page Design in HTML templates

For users to have a good experience navigating different pages, the hyperlink HTML tag `<a href="urls" > </a >` is used for the header, footer and the homepage buttons, allowing user to click on them. The header allows quick navigation to different page and the footer makes it easy for users to navigate without scrolling back to the top.

2.3 Data Log Page

To store the environmental data collected by IoT sensors, a model “Event” is created.

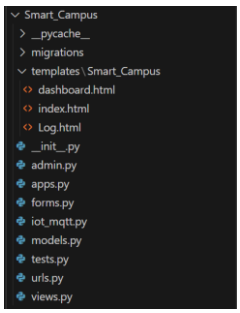

Figure 11: Event model

It stores the node id, location, temperature, relative humidity, light intensity, sound level and the created date of corresponding data, and saves it into the database.

Logic Explains – views.py

The Log(request) function in views.py handles a HTML request for the data log page.

First, it extracts all sensor data from the database, ordered by created date in descending order, and stores it in a list named List. After that, it checks the request method. If it is a POST request, it will extract user input filters from the “Time_DataForm” form, and the list will be filtered based on the inputs.

Lastly, it passes the filtered List and form to context, and an HTML response is returned with the context included.

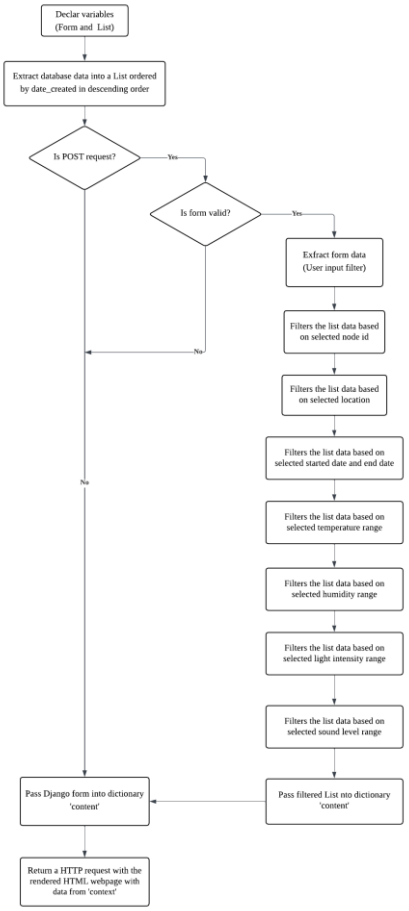

Figure 12: Flow Chart for the Logic of Data Log Page

2.4 Dashboard

The size of pre-processed data in the database can be huge and it is unnecessary to pass the entire database every time accessing the dashboard. Upon receiving a user-specified time interval, an initial **GET request** fetches raw sensor data from the database, which is subsequently refined via a **POST request** applying precise date filters.

This server-side filtering restricts the dataset to a manageable scope for processing. Initial **GET** request retrieves raw data from the database and then Applies date filters through the **POST** request based on the input returned by users. The system then dynamically segments the time axis into contextually appropriate intervals, which is determined by analytical purpose. High-resolution intervals (e.g., **5-minute increments**) reveal operational nuances for performance monitoring (e.g., cooling efficiency of individual air-conditioning unit), while coarser aggregations (e.g., **hourly increments**) better illustrate usage trends across broader timeframes (e.g., daily cooling demand cycles). The resulting visualizations thus deliver charts of different granularity to end-users, depending on analytical purposes.

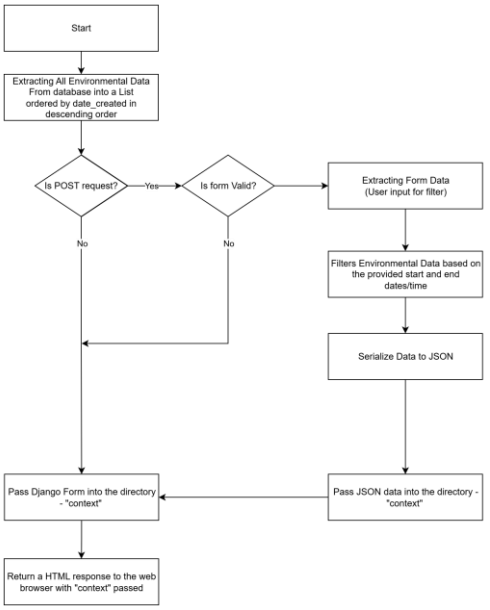

Figure 13: Flowchart of the logic of Dashboard

```
const aggregatedData = {};

eventsData.forEach(item => {
    const fields = item.fields;
    if (fields) {
        const dateCreated = new Date(fields.date_created).toISOString().slice(0, 13) + ":00:00";
        const nodeId = fields.node_id || 'Unknown';
        const loc = fields.loc || 'Unknown';
        const temp = parseFloat(fields.temp) || 0;
        const hum = parseFloat(fields.hum) || 0;
        const light = parseInt(fields.light, 10) || 0;
        const snd = parseInt(fields.snd, 10) || 0;
```

Figure 14: Intermediate structure for each interval

```
// Create an array of time intervals
const timeIntervals = [];
const startTime = new Date(Math.min(...eventsData.map(item => new Date(item.fields.date_created))));
const endTime = new Date(Math.max(...eventsData.map(item => new Date(item.fields.date_created))));

for (let dt = startTime; dt <= endTime; dt.setHours(dt.getHours() + 1)) {
    timeIntervals.push(dt.toISOString().slice(0, 13) + ":00:00");
}
```

Figure 15: Generating timeline ensures consistent x-axis spacing.

```
if request.method == 'POST':
    form = Time_DataForm(request.POST)
    if form.is_valid():
        start_date = form.cleaned_data.get('start_date')
        end_date = form.cleaned_data.get('end_date')
        loc = form.cleaned_data.get('loc')
        node_id = form.cleaned_data.get('node_id')

        if start_date:
            events = events.filter(date_created__gte=start_date)

        if end_date:
            events = events.filter(date_created__lte=end_date)
```

Figure 16: Filter of start and end times

Pre-Plotting Data Processing

Due to varying sampling rates and irregular time intervals in sensor readings, data segmentation into fixed time intervals is necessary. This implementation aggregates reading into hourly intervals. The total number of intervals is derived from the user-specified start and end times. An intermediate data structure stores the averaged values of each parameter (temperature, humidity, light, and sound) per interval, normalizing the data to a unified timeframe. This processed data is then structured for visualization using Flexmonster to generate the required graphs.

2.5 Time-Event-Data Page

New Django Application

First, a new Django application is created – Time_event_data

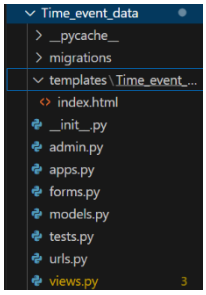

Figure 17 : Time_event_data django application

Storing Venue Events

In order to store the venue events data, a Django model is created, storing the Venue/location, starting time, ending time, event code and the instructor.

```
class Venue(models.Model):
    Venue = models.CharField(max_length=10,blank = False)
    Time_start = models.DateTimeField(auto_now_add=False,blank = False)
    Time_end = models.DateTimeField(auto_now_add=False,blank = False)
    Event = models.CharField(max_length=10,blank = False)
    Instructor  = models.CharField(max_length=20,blank = False)

    def __str__(self):
        return 'Venue Event #{}'.format(self.id)
```

Figure 18: Venue Model

Logic Explains – views.py

The index(request) function in views.py handles HTML requests. It defines multiple variables for venue event data, environmental data statistics for each venue event, and overall environmental data statistics.

If it is a POST request, it extract user input from the Django Form, extract all Venue Event Data from database and filter them accordingly to user input (location and time range), after that a for loop that iterate each event. For each venue, it extracts environmental data records of each venue, further filter out invalid record based on user input and calculates statistics (average, max, min) for temperature, humidity, light, and sound. The statistics are formatted and stored in Each_Venue_Data for each venue events and OVERALL for all venues event combined.

Finally, it includes a Django Form in context and returns an HTML response with context passed to the template.

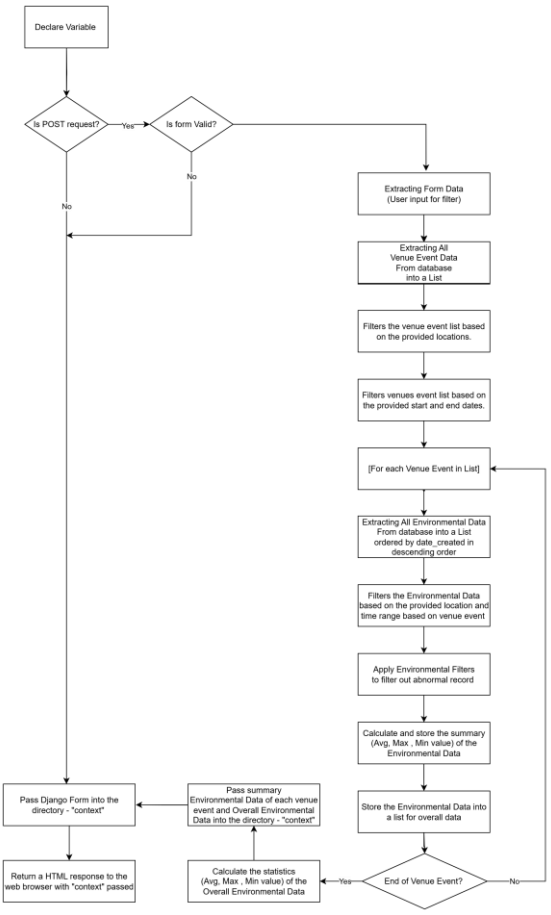

Figure 19: Flow Chart for the Logic of Time-Event-Data Page

2.6 Data analysis and Alert Generations

Logic Explains – Abnormal event defined

The time range for abnormal events detection is between 11:00 pm to 6:00 am and are determined based on certain thresholds for temperature, light, and sound levels as following:

Parameter	Threshold	Description/Explain
Temperature	>= 30 or <= 10	Events with temperatures above 30°C or below 10°C are considered abnormal
Light Intensity	>= 30	Events with light Intensity of 30% or above are considered abnormal
Sound Level	>= 65	Events with sound levels of 65 Db or above are considered abnormal

Logic Explains – views.py

The index(request) function in views.py handles HTML requests. First, it extracts all environmental data from the database, ordered by the creation date in descending order, and stores it in a list called Abnormal_events.

Next, it applies a filter to include only records from 11:00 PM to 6:00 AM. After that, three lists are created, querying abnormal data based on specific thresholds. The relative time since the current time for each record is then calculated.

Finally, the information/records are passed into a dictionary as context and an HTML response is returned to the web browser with the context included.

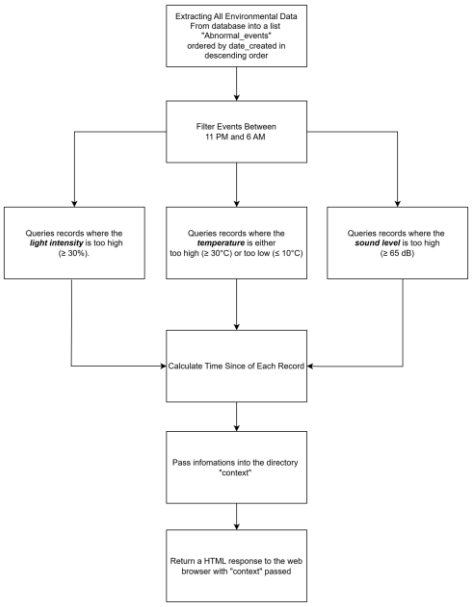

Figure 20: Flow Chart for the Logic of Data

```python
from django.shortcuts import render
from Smart_Campus.models import Event
import datetime
from django.utils.timesince import timesince
from django.db.models import Q

# Create your views here.
def index(request):

    Abnormal_events = Event.objects.order_by('-date_created') # date_created descending o
    Abnormal_events = Abnormal_events.filter(
    Q(date_created__hour__gte=23) | Q(date_created__hour__lte=6))
    Abnormal_events_temp = Abnormal_events.filter(Q(temp__gte = 30) | Q(temp__lte = 10))
    Abnormal_events_light = Abnormal_events.filter(light__gte = 30)
    Abnormal_events_snd = Abnormal_events.filter(snd__gte = 65)

    temp_time = []
    light_time = []
    snd_time = []


    for event in Abnormal_events_temp:
        temp_time.append(timesince(event.date_created))

    for event in Abnormal_events_light:
        light_time.append(timesince(event.date_created))

    for event in Abnormal_events_snd:
        snd_time.append(timesince(event.date_created))

    Abnormal_events_temp = zip(Abnormal_events_temp,temp_time)
    Abnormal_events_light = zip(Abnormal_events_light,light_time)
    Abnormal_events_snd = zip(Abnormal_events_snd,snd_time)

    context = {'Abnormal_events':{'temp':Abnormal_events_temp,
               'light': Abnormal_events_light,
               'snd' : Abnormal_events_snd,}
            }
```

Figure 21: Data_analysis/view.py

3) Analysis of the IOT data and performance result

3.1 IOT Data

We created a SQL table in database for analysis of IoT data. Before creating table, we had deleted all dummy variables written for presentation of the project to avoid invalid or impossible values (e.g. temperature with 200°C).

	loc	node_id	avg_temp	max_temp	min_temp	avg_hum	max_hum	min_hum	avg_light	max_light	min_light	avg_snd	max_snd	min_snd
1	W311-H1	A03	22.45	23.0	21.0	57.47	66.0	53.0	22.21	65.0	0.0	-60.69	-24.0	-99.0
2	W311-H2	A04	25.76	28.0	24.0	62.75	73.0	57.0	15.3	42.0	0.0	-63.45	-41.0	-99.0
3	W311-H3	A05	25.64	27.0	24.0	46.38	53.0	43.0	20.31	52.0	1.0	-56.27	-35.0	-99.0
4	W311A	A01	22.1	25.0	18.0	56.22	71.0	47.0	18.89	63.0	0.0	-54.68	-27.0	-99.0
5	W311B	A02	25.03	27.0	22.0	56.15	68.0	51.0	18.36	79.0	0.0	-62.69	18.0	-99.0
6	W311D-Z1	A06	23.49	24.0	10.0	49.82	54.0	45.0	19.77	39.0	1.0	-60.5	100.0	-99.0
7	W311D-Z2	A07	23.38	40.0	22.0	54.28	63.0	48.0	53.5	86.0	2.0	-60.35	-41.0	-99.0

Figure 22: SQL table of IoT data

According to the table, the maximum temperature of 40°C in W311D-Z2 among all venues, while the minimum temperature was 10°C in W311D-Z1. The highest humidity was 73.0% in W311-H2, and the lowest humidity was 43.0% in W311-H3. Meanwhile, the highest light intensity level was 86.0% in W311D-Z2 and highest sound level was 100dB in W311D-Z1.

3.2 Performance

Data Filtering and Accuracy

We believe that our web application is performing well. First, we are pleased with the performance of our web application's filter on the Log page, Dashboard page, and Time-Event-Data page, as it successfully applies filters and displays only the information that meets the filter criteria.

User Experience on Navigation

Our website design is consistent as we used same Bootstrap CSS for HTML frameworks. The content layout is well-organized. Visible navigations are also provided to improve website usability.

Loading Speed

Most pages load quickly, but the dashboard may take longer to load due to having over 3,000 records. Fortunately, it can successfully present the IoT data records that match the optional filter criteria in a chart format, and the loading speed remains acceptable.

Individual Section

1) Kit

As a part of Group 8, I am mainly responsible for the Time-Event-Data page and the data analysis and alert generation aspects of the project. I also worked on web design and the interaction between the Raspberry Pi and the IoT status tag. There were a few memorable challenges during the development process.

First, regarding the Time-Event-Data page, I need a form for users to select the venue/location and time range to extract venue events, with an optional filter for environmental data records. Considering that venue events and environmental data records are two Django models with different attributes, I have decided to use a Django form instead of a Django model form. This allows me to create a form that is not tied to a specific model, providing more flexibility. I have created a Multiple Choice Field for selecting the venue/location which options are determined by extracting all venues from the database, making it a dynamic design. After gathering the user input from the form, the next step is to implement the logic to extract information from the database based on user input and display them, as explained in 2.5.

Next, for the data analysis and alert generation page, my main task is to use the filter function to query records that are considered abnormal. In order to calculate the relative time since the current time for each record, allowing the user to know how recent the records are, I imported the timesince function from django.utils.timesince. This function takes two datetime objects and calculates the time difference between them. If only one datetime object is provided, it calculates the time since the current time.

In conclusion, from this project, I further understand that Django as a web framework is powerful and provides a lot for web applications development. Functions and tools mentioned in the previous module are only a portion of what Django provides. I learned the importance of solving problems through research and reading documentation to find, understand and use the right tools. In this case, I successfully discovered and utilized the timesince function and Django forms to tackle the problems I faced. As there are many libraries and tools made by other developers and new tools will continue to be developed. It is natural for school to only focus on teaching a select few that are important. I cannot rely solely on what I learned in school to solve specific problems in software development. Rather than just memorizing, it would be more important for me to develop the ability to search for the right tools, understand their functionality, and apply them using critical thinking skills so I can adapt and solve software development challenges in my future career.

2) [Group Member 2]

In this project, I am responsible for configuring the M5-StickCPlus2 sensor and assisting with the implementation of applications on the Raspberry Pi. It involved setting up sensor nodes to collect data and to perform different tasks, and ensuring the data could be successfully transmitted into database for further processing and analysis.

Throughout the project, I have encountered several challenges. First, for the M5-StickCPlus2 configuration, my task is to make it an additional IoT sensor, such that I need to convert the measured data into JSON format and transmit them to MQTT broker. Therefore, I imported ArduinoJson library to serialize and deserialize JSON data in Arduino. However, after I performed data serialization, I found that some data contains NULL. To solve this problem, I increased the capacity of JSON document to provide sufficient size for storing data.

Second, for the alert system of M5Stick, I need to understand its features and specifications first, including built-in sensors and hardware functionality. This required me to do research about M5Stick. I read the device's online documentation and conducted several hand-on experiments to test its functionality. By understanding its techniques and behaviors, I was able to configure the alert systems to detect abnormalities and give warning to users.

To conclude, this project gave me a valuable opportunity to build products in IoT applications. By utilizing open-source libraries and tools, I can configure the M5-StickCPlus2 sensor and Raspberry Pi, and integrate them into the IoT framework. I have also learnt practical knowledge of using Arduino to design microcomputer applications and develop IoT products. Moreover, my problem-solving, technical research and troubleshooting skills are also enhanced through hands-on exercises. I can identify root causes of problems and implement solutions systematically. This project has equipped me with essential skills and knowledge for IoT application development, which will be beneficial for my future career in IoT and related fields.

3) [Group Member 3]

I am mainly responsible for the dashboard section of this project.

A dashboard functions as a visual display of the most important information, consolidated and arranged on a single screen, to provide a clear and immediate overview of performance, status, or key metrics. It is used to monitor and track real-time data on key performance indicators (KPIs), metrics, and trends. It also helps users quickly identify patterns, spot anomalies, compare performance, and understand the current situation. Based on these two main purposes, the data should be presented in an easily digestible format (like charts, graphs, gauges, and tables), enabling users to make informed decisions faster and take necessary actions.

My first methodology centered on transferring the entire pre-processed database directly to Flexmonster for client-side filtering. This approach proved fundamentally flawed due to three critical oversights. First of all, the approach was resource inefficient. The database's scale (potentially thousands of records) rendered full transfers computationally prohibitive, consuming excessive bandwidth and memory for each request. Secondly, the result was statistical irrelevance. Raw sensor data collected at inconsistent intervals lacked inherent statistical significance when visualized without normalization. Thirdly, Flexmonster's internal data truncation mechanisms activated unexpectedly with large datasets, silently discarding records and producing misleading visual outputs. The key realizations are the necessity of pre-aggregation and contextual filtering. To begin with, Client-side tools like Flexmonster should receive purpose-built datasets, not raw repositories, to avoid implicit quality compromises. Next, user-specific time ranges must restrict data retrieval at the database level, not during visualization. Eventually, Statistical meaning emerges only through data segmentation and measurements should be transformed into comparable units. The simplest example is to define the x-axis as hourly averaging is much more useful than logging time.

The foremost challenge in dashboard development arose from Flexmonster's inherent constraints, despite its strengths as a visualization engine. While proficient at multidimensional analysis, the tool proved inadequate for sensor analytics due to its daily-minimum time filtering granularity, which fail to fulfil the requirement of the checkpoint. To resolve this fundamental limitation, I decide to restrict Flexmonster to data rendering only, handling all temporal

processing externally. Raw events are chronologically sorted using deterministic timestamp parsing. Then, server-side windowing isolates user-specified time ranges at database level. After that, the data are grouped into configurable blocks (e.g., 15-min/hourly).

This project is a memorable learning experience. tool limitations became an unexpected catalyst for deeper exploration. It gives me an incentive to deep research into specialized principles of IoT data presentation for end-users, particularly granularity optimization and the importance of a well-designed database hierarchy This project has fundamentally reshaped my understanding of time-series data architecture, transforming preprocessing from a technical necessity into the core foundation of user-centric design.